Ejemplo completo de uso de `.github`

Resumen

Este documento muestra, con ejemplos completos y realistas, cómo se usa lo definido en la carpeta ../../.github de este repositorio.

La idea no es describir GitHub en abstracto, sino explicar cómo se trabaja aquí con:

- políticas base de Copilot
- agentes especializados de VS Code
- instrucciones por tipo de archivo
- prompts reutilizables
- plantillas de issue y pull request
- validación CI
- gobernanza del repositorio

Primeros pasos

Jerarquía de decisión

El orden efectivo de aplicación está resumido en ../../.github/README.md:

Nivel	Archivo o carpeta	Para qué sirve	Cuándo manda
1	../../.github/copilot-instructions.md	Política transversal del repositorio	Siempre
2	../../.github/agents	Roles especializados con límites de herramientas y responsabilidades	Cuando se delega o selecciona un agente
3	../../.github/instructions	Reglas por ruta o tipo de archivo	Cuando el archivo coincide con `applyTo`
4	../../.github/prompts	Flujos de trabajo para tareas concretas	Cuando se invoca el prompt
5	../../.github/ISSUE_TEMPLATE, ../../.github/pull_request_template.md, ../../.github/workflows/tests.yml	Operativa de GitHub	Al abrir issues, PRs o ejecutar CI

Regla práctica

- Si vas a pedir trabajo al agente, empieza por la política base.
- Si la tarea necesita separar planificación, implementación o auditoría, usa además el agente especializado correspondiente.
- Si el cambio cae en una ruta concreta, además aplican las instrucciones de esa ruta.
- Si quieres forzar un formato de salida o un modo de trabajo, usa el prompt correspondiente; el prompt define la salida y el agente define el rol.
- Si el cambio se va a revisar o registrar en GitHub, usa las plantillas y la validación definidas.

Configuración

La carpeta ../../.github ya deja configurados estos comportamientos:

Elemento	Archivo	Configuración relevante
Política base	../../.github/copilot-instructions.md	Senior Python, librería multibase, salida estructurada, políticas por tipo de trabajo
Agentes	../../.github/agents	Routing, planificación, implementación, hotfix, auditoría, seguridad y cierre
Instrucciones docs	../../.github/instructions/docs.instructions.md	Markdown portable, ejemplos claros, secciones consistentes
Instrucciones Python	../../.github/instructions/python.instructions.md	Python 3.12+, tipado, separación introspect/render
Instrucciones introspección	../../.github/instructions/introspection.instructions.md	Multibase, introspección agnóstica, límites por motor
Instrucciones render	../../.github/instructions/render.instructions.md	Salida determinista y render puro
Instrucciones render Markdown	../../.github/instructions/render-markdown.instructions.md	Orden de secciones y formato por tabla
Instrucciones tests	../../.github/instructions/tests.instructions.md	Tests pequeños, deterministas y sin red
Prompts	../../.github/prompts	Flujos estandarizados para feature, hotfix, refactor, revisión y cierre
Issues	../../.github/ISSUE_TEMPLATE	Formularios para bug y feature
PR	../../.github/pull_request_template.md	Resumen, alcance, validación y riesgos
Seguridad	../../.github/SECURITY.md	Reporte privado y alcance prioritario
Contribución	../../.github/CONTRIBUTING.md	Flujo mínimo y validación local
Ownership	../../.github/CODEOWNERS	Responsable por áreas
Dependabot	../../.github/dependabot.yml	Actualizaciones semanales de pip y GitHub Actions
CI	../../.github/workflows/tests.yml	Windows y Ubuntu, Python 3.12 a 3.14

Agentes disponibles

La carpeta ../../.github/agents añade una capa de especialización operativa sobre la política base y los prompts.

Agente	Archivo	Úsalo cuando

Agente	Archivo	Úsalo cuando
`Workflow Director`	../../.github/agents/workflow-director.agent.md	Necesitas clasificar una petición y decidir skill, fase o flujo
`Plan Architect`	../../.github/agents/plan-architect.agent.md	Necesitas construir `PLAN` y `AUDIT_PLAN` contra el código real
`Feature Implementer`	../../.github/agents/feature-implementer.agent.md	Ya existe alcance aprobado y hay que ejecutar `IMPLEMENT`
`Hotfix Responder`	../../.github/agents/hotfix-responder.agent.md	Hay que aplicar un fix mínimo con RCA y rollback
`Code Auditor`	../../.github/agents/code-auditor.agent.md	Quieres un `AUDIT_CODE` o revisión de regresiones
`Security Reviewer`	../../.github/agents/security-reviewer.agent.md	Quieres una auditoría de seguridad independiente
`Version Closer`	../../.github/agents/version-closer.agent.md	Necesitas cerrar una sesión, release o bloque de cambios

Uso

Caso 1. Política base siempre activa

Cuando pides un cambio cualquiera, la política de ../../.github/copilot-instructions.md fija el marco:

- hay que responder en Markdown estructurado
- no se deben inventar módulos, clases o tests
- hay que emitir PLAN y AUDIT_PLAN antes del resto cuando el trabajo lo requiera
- `new_feature`, `hotfix`, `refactor` y `security_review` tienen restricciones distintas

Ejemplo de petición:

```
Necesito añadir documentación Markdown para vistas y relaciones FK en el renderer.
```

Efecto esperado:

- el agente debe identificar que es una feature
- debe separar plan, riesgos, implementación y cierre
- no debe mezclarlo con un refactor no solicitado

Caso 2. Instrucciones activadas por ruta

Las instrucciones de ../../.github/instructions se activan por patrón `applyTo`.

Si editas	Se activa	Qué añade

Si editas	Se activa	Qué añade
`docs/**`	../../.github/instructions/docs.instructions.md	Markdown portable, secciones consistentes, ejemplos claros
`**/*.py`	../../.github/instructions/python.instructions.md	Python 3.12+, tipado, API estable
`src/**/introspect/**/*.py`	../../.github/instructions/introspection.instructions.md + reglas Python	Nada de asumir un solo motor, adaptar por dialecto
`src/**/render/**/*.py`	../../.github/instructions/render.instructions.md + ../../.github/instructions/render-markdown.instructions.md + reglas Python	Orden determinista, formato consistente, Markdown real
`tests/**`	../../.github/instructions/tests.instructions.md	Tests pequeños, deterministas, reproducibles

Ejemplos rápidos:

2.1 Editar documentación

Archivo objetivo:

```
docs/examples/nuevo_ejemplo.md
```

Qué debe pasar:

- usar Markdown estándar
- incluir ejemplos sin credenciales reales
- evitar HTML embebido si no es imprescindible

2.2 Editar introspección

Archivo objetivo:

```
src/util_bbdd/algun_modulo/introspect/sqlite.py
```

Qué debe pasar:

- no asumir que SQLite representa a todos los motores
- documentar limitaciones reales
- encapsular diferencias por motor

2.3 Editar render Markdown

Archivo objetivo:

```
src/util_bbdd/algun_modulo/render/markdown.py
```

Qué debe pasar:

- mantener salida determinista
- respetar orden de secciones por tabla
- usar Mermaid solo si deriva de metadatos reales y aporta valor claro

2.4 Editar tests

Archivo objetivo:

```
tests/test_markdown_renderer.py
```

Qué debe pasar:

- usar fixtures pequeñas
- asserts sobre fragmentos clave o snapshots controlados
- no depender de red ni servicios externos obligatorios

Caso 3. Uso de agentes

Los agentes de ../../.github/agents no sustituyen a los prompts. Su función es aislar responsabilidad, limitar herramientas y hacer explícita la fase de trabajo.

Cadena recomendada en este repositorio:

```
Workflow Director -> Plan Architect -> Feature Implementer o Hotfix Responder -> Code
Auditor -> Version Closer
```

Regla práctica:

- usa `Workflow Director` cuando la petición todavía no esté clasificada
- usa `Plan Architect` antes de implementar una feature o un hotfix relevante
- usa `Feature Implementer` solo cuando el alcance ya está aprobado
- usa `Hotfix Responder` para incidencias con fix mínimo
- usa `Code Auditor` para revisar regresiones, contratos y cobertura
- usa `Security Reviewer` cuando el objetivo principal sea seguridad
- usa `Version Closer` al cerrar una versión, PR o sesión de trabajo

Tabla de mapeo rápido entre prompts y agentes:

Tipo de tarea	Prompt habitual	Agente recomendado
Clasificar una petición ambigua	opcional	`Workflow Director`
Planificar una feature	`new_feature`	`Plan Architect`

Tipo de tarea	Prompt habitual	Agente recomendado
Implementar una feature ya aprobada	`new_feature`	`Feature Implementer`
Corregir un bug acotado	`hotfix`	`Hotfix Responder`
Refactorizar sin cambio funcional	`refactor`	`Plan Architect` y luego `Feature Implementer`
Auditar código	`audit_code`	`Code Auditor`
Revisar seguridad	`security_review`	`Security Reviewer`
Inventariar deuda técnica	`tech_debt`	`Code Auditor`
Evaluar madurez	`maturity_assessment`	`Code Auditor`
Cerrar una versión o sesión	`version_closure`	`Version Closer`

Caso 4. Uso de prompts

Los prompts de ../../.github/prompts sirven para encauzar tareas frecuentes con una salida fija.

En este repositorio pueden invocarse como símbolos en chat, por ejemplo:

```
@#sym:new_feature
@#sym:hotfix
@#sym:version_closure
```

Tabla resumen:

Prompt	Archivo	Úsalo cuando	Salida esperada
`new_feature`	../../.github/prompts/new_feature.prompt.md	Vas a añadir comportamiento nuevo	`PLAN`, `AUDIT_PLAN`, `IMPLEMENT`, `AUDIT_CODE`, `VERSION`
`hotfix`	../../.github/prompts/hotfix.prompt.md	Hay que corregir un bug acotado	`PLAN`, `AUDIT_PLAN`, `IMPLEMENT`, `AUDIT_CODE`, `VERSION`
`refactor`	../../.github/prompts/refactor.prompt.md	Quieres mejorar estructura sin cambiar comportamiento	`PLAN`, `AUDIT_PLAN`, `IMPLEMENT`, `AUDIT_CODE`, `VERSION`

Prompt	Archivo	Úsalo cuando	Salida esperada
`security_review`	../../.github/prompts/security_review.prompt.md	Quieres revisar seguridad	`PLAN`, `AUDIT_PLAN`, `AUDIT_CODE`, `VERSION`
`audit_code`	../../.github/prompts/audit_code.prompt.md	Quieres una revisión técnica de código	`PLAN`, `AUDIT_PLAN`, `AUDIT_CODE`, `VERSION`
`tech_debt`	../../.github/prompts/tech_debt.prompt.md	Quieres inventariar deuda técnica	`PLAN`, `AUDIT_PLAN`, `AUDIT_CODE`, `VERSION`
`maturity_assessment`	../../.github/prompts/maturity_assessment.prompt.md	Quieres valorar madurez del repositorio	`PLAN`, `AUDIT_PLAN`, `AUDIT_CODE`, `VERSION`
`version_closure`	../../.github/prompts/version_closure.prompt.md	Quieres cerrar una versión o un bloque de cambios	commit message, changelog, impacto, rollback y checklist

Relación entre prompt y agente:

- el prompt fija el contrato de salida
- el agente fija la fase, el rol y el nivel de autonomía
- cuando ambos se usan juntos, el resultado es más predecible y más fácil de revisar

Ejemplos

Ejemplo completo 1. `new_feature`

Invocación:

```
@#sym:new_feature

Objetivo: añadir exportación Markdown de vistas con Mermaid opcional.
Módulo(s) o clase(s) afectadas: src/util_bbdd/doc/render_markdown.py,
tests/test_render_markdown.py
API pública esperada: parámetro opcional para habilitar diagramas Mermaid
Contrato de entrada: modelo de metadatos ya introspectado
Contrato de salida: Markdown portable y determinista
Reglas de negocio: no inventar metadatos, Mermaid solo con datos reales
Restricciones: sin dependencias nuevas
Motores o formatos afectados: Markdown
```

```
Migraciones necesarias: ninguna
Archivos existentes: sí
```

Qué obliga este prompt:

- presentar `PLAN`
- presentar `AUDIT_PLAN`
- implementar por capas o rutas
- cerrar con `AUDIT_CODE` y `VERSION`
- no convertir la tarea en refactor general

Agente recomendado:

- `Plan Architect` para construir `PLAN` y `AUDIT_PLAN`
- `Feature Implementer` solo después de aprobar alcance
- `Code Auditor` para la revisión posterior

Ejemplo completo 2. `hotfix`

Invocación:

```
@#sym:hotfix

Bug: el renderer HTML pierde iconos offline cuando se exporta fuera del workspace.
Rutas afectadas: src/util_bbdd/doc/render_html.py, tests/test_html_assets.py
Dato faltante o supuesto: se asume que la salida debe ser portable sin acceso a red
```

Qué obliga este prompt:

- arreglar solo la causa raíz
- no refactorizar
- no optimizar fuera del bug
- añadir test de reproducción si es viable

Agente recomendado:

- `Plan Architect` si todavía falta delimitar impacto
- `Hotfix Responder` para ejecutar el fix mínimo

Ejemplo completo 3. `refactor`

Invocación:

```
@#sym:refactor

Objetivo: separar construcción de payload de leyenda y render HTML sin cambiar
comportamiento.
Módulos: src/util_bbdd/doc/diagram_legend.py, src/util_bbdd/doc/render_html.py
Contrato público: sin cambios
Restricciones: preservar snapshots y orden actual de salida
```

Qué obliga este prompt:

- no introducir cambios funcionales
- priorizar responsabilidad única y legibilidad
- justificar cualquier cambio en API pública

Agente recomendado:

- `Plan Architect` para delimitar alcance y riesgos
- `Feature Implementer` para ejecutar el refactor aprobado

Ejemplo completo 4. `security_review`

Invocación:

```
@#sym:security_review

Revisa la generación documental para detectar fuga de secretos, exposición de SQL
sensible e inyección por identificadores.
Áreas: src/util_bbdd/doc, src/util_bbdd/analysis, docs/examples
```

Qué obliga este prompt:

- ordenar hallazgos por severidad
- separar confirmados de supuestos
- dar remediación accionable
- no mezclarlo con una feature

Agente recomendado:

- `Security Reviewer`

Ejemplo completo 5. `audit_code`

Invocación:

```
@#sym:audit_code

Revisa src/util_bbdd/doc/render_markdown.py y src/util_bbdd/doc/render_html.py.
Quiero bugs, edge cases, compatibilidad y tests faltantes.
```

Qué obliga este prompt:

- emitir veredicto final
- citar evidencias concretas por archivo o módulo
- diferenciar hallazgos confirmados de hipótesis

Agente recomendado:

- `Code Auditor`

Ejemplo completo 6. `tech_debt`

Invocación:

```
@#sym:tech_debt

Analiza la deuda técnica de src/util_bbdd/doc y src/util_bbdd/analysis.
Quiero quick wins, prioridades y plan por fases.
```

Qué obliga este prompt:

- listar deuda por módulo
- incluir impacto y prioridad
- proponer quick wins y plan por fases

Agente recomendado:

- `Code Auditor`

Ejemplo completo 7. `maturity_assessment`

Invocación:

```
@#sym:maturity_assessment

Evalúa la madurez del repositorio para una publicación estable centrada en
documentación multibase y render HTML offline.
```

Qué obliga este prompt:

- emitir veredicto de madurez
- separar bloqueantes de mejoras recomendadas
- revisar APIs, contratos, seguridad, tests y mantenibilidad

Agente recomendado:

- `Code Auditor`

Ejemplo completo 8. `version_closure`

Invocación:

```
@#sym:version_closure
```

Uso típico:

- después de alinear metadatos, CI y documentación
- al finalizar una release o un bloque coherente de cambios
- para preparar commit message, changelog, impacto, compatibilidad y rollback

Salida mínima esperada, según ../../.github/prompts/version_closure.prompt.md:

- commit message
- changelog
- impacto en API pública
- migraciones
- compatibilidad
- rollback
- checklist final
- pruebas ejecutadas o sugeridas

Agente recomendado:

- `Version Closer`

Ejemplo completo 9. Crear una issue de bug

La plantilla está en ../../.github/ISSUE_TEMPLATE/bug_report.md.

Ejemplo de issue:

```
## Descripción
La exportación HTML offline no copia los iconos SVG del toolbar y deja botones sin
imagen.

## Módulo, clase o función afectada
src/util_bbdd/doc/render_html.py

## Pasos para reproducir
1. Generar documentación HTML offline.
2. Abrir index.html fuera del workspace.
3. Ir a la vista global "Ver todo".

## Resultado esperado
Los iconos se muestran correctamente sin acceso a red.

## Resultado actual
Los botones aparecen sin icono.

## Motor, entorno y versión
SQLite, Windows 11, util_bbdd 0.6.2, Python 3.12

## Configuración implicada
Salida HTML offline con assets locales.

## Riesgo
La documentación exportada pierde claridad visual y parece incompleta.

## Evidencia mínima
Captura de la página generada y listado de assets ausentes.
```

Ejemplo completo 10. Crear una issue de feature

La plantilla está en ../../.github/ISSUE_TEMPLATE/feature_request.md.

Ejemplo de issue:

```
## Objetivo
Permitir exportar diagramas de relaciones en Mermaid además de Graphviz para Markdown.

## Contexto
Algunos consumidores publican documentación en renderers Markdown sin soporte SVG
incrustado.

## Módulo, clase o área afectada
src/util_bbdd/doc/render_markdown.py

## API pública esperada
Nuevo parámetro opcional `diagram_format="graphviz" | "mermaid"`.

## Motores o formatos afectados
Markdown

## Criterios de aceptación
- salida determinista
- sin inventar relaciones
- tests con metadatos reales o fixtures mínimas

## Riesgos
Duplicar lógica de render si no se separa bien el payload común.

## Restricciones o supuestos
No introducir servicios externos ni dependencias pesadas.
```

Ejemplo completo 11. Abrir una PR

La plantilla está en ../../.github/pull_request_template.md.

Ejemplo de PR rellenada:

```
## Resumen
Se alinea el soporte oficial a Python 3.12+ y se sincronizan CI, metadatos y
documentación.

## Tipo de cambio
- [x] hotfix

## Rutas afectadas
- pyproject.toml
- README.md
- .github/workflows/tests.yml
- docs/plan/**

## Impacto en API pública y compatibilidad
No cambia la API Python, pero el runtime soportado pasa a 3.12+.

## Supuestos o datos faltantes
Se asume que no se dará soporte oficial a 3.11 en adelante.

## Validación
```

```
- [x] revisión manual
- [ ] lint ejecutado
- [ ] tests ejecutados
- [ ] type-check ejecutado
- [x] rollback definido

## Riesgos
Usuarios con entornos 3.11 deberán recrear el runtime.

## Evidencia
Se adjuntan diff de CI, metadatos y README.
```

Ejemplo completo 12. Reporte de seguridad

El punto de entrada está en ../../.github/SECURITY.md y se enlaza desde ../../.github/ISSUE_TEMPLATE/config.yml.

Ejemplo de reporte privado:

```
Asunto: Posible fuga de credenciales en artefactos HTML generados

- Descripción: se detecta que el HTML exportado puede incluir DSN completos en un
bloque de metadatos.
- Impacto estimado: exposición de host, usuario técnico y estructura interna.
- Pasos mínimos: generar documentación con un DSN explícito y revisar el bloque de
cabecera.
- Versiones afectadas: 0.6.1 y 0.6.2
- Mitigación propuesta: sanear configuración sensible antes del render y añadir test de
regresión.
```

Ejemplo completo 13. Flujo de contribución

El flujo está en ../../.github/CONTRIBUTING.md.

Ejemplo de uso correcto:

1. Abrir una rama corta solo para `hotfix`.
2. Revisar ../../.github/copilot-instructions.md y la instrucción específica del área.
3. Aplicar cambio pequeño.
4. Añadir tests.
5. Ejecutar localmente:

```
python -m ruff check src tests
python -m mypy src
python -m pytest
```

6. Abrir PR con plantilla y riesgos explícitos.

Ejemplo completo 14. `CODEOWNERS`

La definición está en ../../.github/CODEOWNERS.

Uso práctico:

- si cambias `src/`, el owner por defecto es `@fmanza`
- si cambias `tests/`, el owner por defecto es `@fmanza`
- si cambias `docs/`, el owner por defecto es `@fmanza`
- si cambias `.github/`, el owner por defecto es `@fmanza`

Ejemplo:

```
Una PR que modifica src/util_bbdd/doc/render_html.py y tests/test_render_html.py
solicitará revisión del owner definido.
```

Ejemplo completo 15. Dependabot

La configuración está en ../../.github/dependabot.yml.

Qué hace:

- revisa semanalmente dependencias `pip`
- revisa semanalmente GitHub Actions
- limita PRs abiertas a 5 por ecosistema
- etiqueta con `dependencies` y, para Actions, también `github-actions`

Ejemplo esperado:

```
Dependabot abre una PR semanal para subir codecov/codecov-action o una dependencia
Python del proyecto.
```

Ejemplo completo 16. CI

La validación está en ../../.github/workflows/tests.yml.

Qué valida hoy:

Sistema	Versiones Python
Ubuntu	3.12, 3.13, 3.14
Windows	3.12, 3.13, 3.14

Qué ejecuta:

1. checkout
2. instalación editable con extras `dev`
3. `ruff`
4. `mypy`
5. `pytest` con cobertura
6. subida de cobertura desde Ubuntu 3.12

Ejemplo de lectura correcta del workflow:

- un fallo de `pytest` rompe el job

- `ruff` y `mypy` están en `continue-on-error: true`, así que informan pero no bloquean por sí solos
- la cobertura se publica una sola vez para evitar duplicados

FAQ

¿Cuándo uso un prompt, un agente o una instrucción?

Usa instrucción cuando la regla depende de la ruta o del tipo de archivo. Usa prompt cuando quieras imponer una forma de trabajo o de salida para una tarea concreta. Usa agente cuando quieras fijar el rol operativo, aislar una fase o limitar el tipo de acciones y herramientas.

¿Los agentes sustituyen a los prompts?

No. En este repositorio los prompts siguen marcando el contrato de salida y los agentes añaden especialización por fase. Lo normal es combinarlos.

¿Qué agente debo usar primero si la petición es ambigua?

Empieza por ../../.github/agents/workflow-director.agent.md. Si la tarea requiere contraste con el código real, el siguiente paso suele ser ../../.github/agents/plan-architect.agent.md.

¿Puedo usar `new_feature` para un bug?

No es lo recomendable. Para un bug acotado usa ../../.github/prompts/hotfix.prompt.md, porque prohíbe refactorizar y optimizar fuera del fix.

¿Puedo meter Mermaid en cualquier Markdown?

No. La política actual permite Mermaid solo cuando aporta valor claro y se genera desde metadatos reales, según ../../.github/instructions/docs.instructions.md y ../../.github/instructions/render-markdown.instructions.md.

¿Qué pasa si una regla de prompt choca con una regla de instrucciones?

Primero manda la política base de ../../.github/copilot-instructions.md. Después, la instrucción aplicable por ruta. El prompt fuerza sobre todo el formato y el flujo de trabajo de la tarea, pero no debería contradecir la política base ni el estado real del repositorio.

¿Qué archivo debo leer primero si soy nuevo en el repositorio?

Empieza por ../../.github/README.md y luego por ../../.github/copilot-instructions.md.

¿Qué archivo debo usar para cerrar una versión?

Usa ../../.github/prompts/version_closure.prompt.md y acompáñalo con revisión de ../../.github/pull_request_template.md, ../../.github/CONTRIBUTING.md y ../../.github/workflows/tests.yml.